

4. 프롬프트 평가 방법론

1. 프롬프트 평가의 목적

프롬프트 엔지니어링에서 응답 품질 평가는 매우 중요한 단계입니다. LLM(대규모 언어 모델)의 응답을 체계적으로 평가하면 모델이 의도한 대로 작동하는지 확인하고 개선 방향을 찾을 수 있습니다. 실제로 강건한 LLM 응용을 개발하려면 출력 품질을 지속적으로 검사하고 측정해야 하지만, 이를 제대로 수행하는 것은 쉽지 않은 과제입니다. 평가를 통해 우리는 다음과 같은 인사이트를 얻을 수 있습니다:

- 현재 프롬프트나 모델 설정으로 생성된 응답의 문제점 파악(사실 오류, 불필요한 내용 등)
- 여러 프롬프트 버전 또는 모델 간 성능 비교를 통한 최적 옵션 선정
- 사용자에게 제공하기 전에 위험 요소나 품질 저하 요소를 사전에 발견(예: 부적절한 톤이나 포맷 오류)
- 지속적 평가로 모델 개선 추이를 추적하고 회귀(regression) 발생 시 조기 감지

결국 평가를 통해 모델 응답 품질을 정량적·정성적으로 판단함으로써, 보다 나은 프롬프트 설계 및 모델 튜닝을 이루고 사용자 경험을 향상시킬 수 있습니다. 이러한 평가 없이는 모델 출력의 개선 방향을 명확히 알기 어렵기 때문에, 프롬프트 결과에 대한 체계적 평가는 필수적입니다.

2. 주요 평가지표 설명

프롬프트 응답을 평가할 때는 여러 가지 기준(metrics)을 고려해야 합니다. 이번 강의에서는 특히 네 가지 핵심 지표에 초점을 맞추어 설명합니다:

- 정확성 (Fact Consistency) – 응답 내용의 사실적 정확도와 일관성
- 일관성 (Role & Tone Consistency) – 지정된 역할이나 톤을 응답이 꾸준히 유지하는 정도
- 표현 톤의 적절성 (Target User Tone Fitting) – 사용자나 상황에 맞는 어조/스타일 사용 여부
- 포맷 및 구조 (Format & Structure) – 요구된 출력 형식과 논리 구조를 제대로 따른 정도

각 지표의 의미와 중요성을 아래에서 상세히 알아보겠습니다.

2.1. 정확성 (Fact Consistency)

정확성은 응답이 사실에 부합하고 오류나 환상(hallucination)이 없는지를 평가합니다. 모델이 사실과 다른 정보를 제시하거나, 아예 근거 없이 내용을 지어내는 것은 큰 감점 요인입니다. LLM의 **사실성(factuality)**이란 신뢰할 수 있는 출처에 기반하여 **정확하고 검증 가능한 정보**를 생성하는 능력을 뜻합니다. 예를 들어, 역사적 사실이나 과학적 데이터 등이 질문에 포함될 때 정확성을 평가하게 됩니다.

LLM 평가에서는 종종 **사실 오류**와 **환각(hallucination)**을 구분하기도 합니다. 사실 오류는 실제 정보와 불일치하는 잘못된 내용을 말하고, 환각은 완전히 근거 없이 **모델이 지어낸 정보**를 말합니다. 예를 들어, 아인슈타인에 대한 질문에 모델이 “아인슈타인이 1932년에 노벨상을 받았다”고 답하면 이는 실제로 1921년 수상인 점에서 **사실 오류**이고, “아인슈타인이 뛰어난 화가이기도 했다”고 답하면 이는 전혀 근거 없는 **환각 정보**입니다. 이러한 오류나 환각 여부를 면밀히 살펴 **응답의 사실 일치도**를 평가해야 합니다.

정확성 평가를 위해서는 응답 내용을 **사실 검증**(예: 신뢰 가능한 자료와 대조)하거나, 질문에서 요구한 정보가 제대로 언급되었는지 확인합니다. 이 지표는 잘못된 정보 전달로 인한 사용자 피해를 막고 모델에 대한 **신뢰성 확보**를 위해 가장 우선시되는 평가 요소입니다.

2.2. 일관성 (Role & Tone Consistency)

일관성은 프롬프트에서 지시한 역할(role)이나 톤(tone)을 응답 내내 유지하는 정도를 평가합니다. 많은 챗봇 응용에서 프롬프트에 **특정 역할**(예: “당신은 의사처럼 답변하세요” 또는 “친근한 튜터로서 설명하세요”)이나 말투를 지정하는데, **우수한 응답**이라면 대화가 진행되는 동안 그 **설정**에 **충실**해야 합니다.

예를 들어 사용자 프롬프트에서 “당신은 역사 학자입니다...”라고 역할을 지시했다면, 응답 내용과 말투가 실제 **역사 학자처럼 전문적이고 격식 있게** 일관되어야 합니다. 만약 중간에 어조가 격식체에서 반말로 바뀌거나, 캐릭터 설정을 잊고 일반 AI 어시스턴트처럼 말한다면 **일관성이 낮아지는 것**입니다.

- **톤 일관성(Tone Consistency)**이라는 용어로도 설명하는데, 이는 **응답이 처음에 정해진 캐릭터나 말투의 패턴을 잘 따라가는지** 보는 것입니다. 다시 말해 **지정된 말투**(예: 해적 말투를 쓰라고 했으면 “~하노라” 같은 catchphrase 사용)대로 끝까지 가는지 평가합니다. 이 지표는 사용자에게 **일관된 경험 제공**과 깊은 몰입감을 주기 위해 중요합니다. 특히 튜터봇처럼 역할에 몰입해야 하는 시나리오에서는 일관성이 떨어지면 사용자 입장에서 어색함이나 신뢰 저하로 이어질 수 있습니다.

평가 방법으로는 응답 내용을 처음부터 끝까지 읽으며 말투, 인칭, 어휘 선택이 프롬프트 지시에 어긋남이 없는지 체크합니다. 또한 대화형 응용에서는 **대화 흐름 전후 맥락**에서 톤이 일치하는지도 봅니다. 일관성이 높을수록 응답 품질이 좋다고 판단합니다.

2.3. 표현 톤의 적절성 (Target User Tone Fitting)

표현 톤의 적절성은 응답의 어조(tone)나 스타일이 대상 사용자나 사용 목적에 맞게 잘 조정되었는가를 평가합니다. 예를 들어, 어린이를 위한 응답이라면 쉽고 친절한 말투여야 하고, 전문 지식인 대상이라면 격식 있고 전문 용어를 정확히 쓰는 식입니다. 요청에 명시적으로 "비전문가도 이해할 수 있게 설명해줘"와 같은 톤 지시가 있었다면 이를 제대로 따랐는지가 평가 포인트입니다.

특히 목표 독자나 연령대에 어울리는 표현인가가 중요합니다. 예를 들어, 8세 어린이 대상 콘텐츠라면 언어를 단순화하고 톤을 부드럽게 유지하며 부적절하거나 어려운 주제는 피하는 것이 바람직합니다. 반대로 전문 연구자 대상 보고서라면 지나치게 가벼운 어투는 부적절할 것입니다.

이 지표를 평가하려면 우선 요구 사항 분석이 필요합니다. 프롬프트에서 특정 톤이나 대상이 언급된 경우 그에 맞는지 확인하고, 없더라도 맥락상 예상되는 사용자(예: 고객 상담 시 일반 고객, 튜터봇이면 학생 등)에 맞는 어조인지 살핍니다. 부적절한 속어 사용, 지나친 격식체 등 상황에 어울리지 않는 표현이 있으면 감점 요소가 됩니다. 톤의 적절성은 사용자 만족도와 신뢰에 직결되므로, 평가 시 세심한 주의가 필요합니다.

2.4. 포맷 및 구조 (Format & Structure)

포맷 & 구조 평가는 응답이 요구된 형식에 맞게 작성되었는지와 내용 전개의 논리적 구조가 적절한지를 살핍니다. 프롬프트에 특정 출력 포맷 (예: "표로 답변해줘", "JSON으로 출력해줘", "1, 2, 3 단계로 나눠줘")을 지시한 경우 이를 정확히 지켰는지가 핵심입니다. 또한 글의 구성 측면에서 머리말-본론-결론의 짜임새가 자연스러운지, 목록이나 번호를 사용하라는 지시가 있으면 제대로 활용했는지도 포함합니다.

평가 시 고려할 세부 요소:

- 형식 준수 여부: 프롬프트 요구대로 마크다운, HTML, JSON, 목록 등의 형식을 지켰는가? 예를 들어 JSON으로 출력하라고 했는데 불필요한 설명이 붙거나 형식 오류가 있으면 감점입니다.
- 구조의 논리성: 정보가 논리적인 순서로 배열됐는가? 중복이나 뒤죽박죽 서술이 없는가?
- 일관된 포맷 사용: 번호 목록이나 표 등을 사용했다면 처음부터 끝까지 일관되게 썼는가? (중간에 스타일이 변하면 감점)

모델 평가지표 중에는 지시된 포맷 준수(adherence to format) 여부를 확인하는 경우가 많습니다. 예를 들어, 평가 시 답변이 요구된 형식에 맞게 작성되었는지, 응답이 과도하게 장황하지 않고 간결성을 유지했는지, 또 예의바르고 공손한지, 개인정보와 같은 민감한 정보는 포함되지 않았는지 등이 체크 요소가 될 수 있습니다. 이러한 포맷/구조 관련 평가지표는 비교적 객관적으로 판단하기 쉬운 편이므로, 자동화된 스크립트나 패턴 매칭으로도 검증 가능하다는 장점이 있습니다.

요약하면, 포맷 및 구조 평가는 **지시한 요구 사항을 정확히 따랐는지와 응답이 보기 좋고 이해하기 쉽게 조직되었는지**를 보는 것입니다. 이 부분이 잘 지켜져야 사용자가 응답을 쉽게 이해하고 활용할 수 있기 때문에, 매우 중요한 평가 기준입니다.

3. 각 평가지표별 판단 기준 설명

이번에는 위에서 소개한 각 평가지표를 **어떻게 구체적으로 평가할 것인지, 그 판단 기준**을 살펴보겠습니다. 평가 방법에는 **루브릭 기반의 객관적 평가**와 **주관적 평가** 두 가지 관점이 있습니다. 또한 필요에 따라 **정량적 평가(점수 부여)**와 **정성적 평가(서술형 피드백)**를 병행할 수도 있습니다. 각각의 방식에 대한 특징과 활용법은 다음과 같습니다.

3.1. 객관적 평가 – 루브릭 기반 점수화

객관적 평가는 평가 기준을 명확히 수치화하거나 예/아니오처럼 **명확한 척도로 정의하여**, 누구든 **일관되게 적용**할 수 있도록 하는 방법입니다. 이를 위해 흔히 **루브릭(rubric)**을 사용합니다. 루브릭이란 각 평가지표에 대해 **몇 점이면 어떤 상태인지**를 상세히 기술한 평가표입니다. 예를 들어 정확성 지표에 대해 **5점은 “사실 오류 전혀 없음”, 4점은 “경미한 사실 오류 1개”, ..., 1점은 “대부분 내용이 사실과 다름”**과 같이 규정하는 것입니다.

실무에서는 **1~5점 척도**를 많이 사용하는데, 이는 평가 기준을 세분화하면서도 사람이 일관되게 판단하기 적절한 범위이기 때문입니다. **0~100처럼 너무 세밀하면 평가자 간 편차가 커지고, 3점 정도로 너무 좁으면 구별력이 떨어집니다. 1-5점 척도를 쓰면 각 점수에 대한 명확한 루브릭을 작성하기 용이하고, 평가자도 비교적 일관되게 점수 부여**를 할 수 있습니다.

루브릭 예시 (정확성 지표에 대한 1~5점 기준):

- **5점: 사실적으로 완벽** – 응답 내 모든 정보가 검증된 사실과 일치함. 오류나 환각 전혀 없음.
- **4점: 대체로 정확** – 주요 정보는 정확하나 아주 사소한 데이터 (숫자 등) 하나 정도 부정확가능.
- **3점: 보통 수준** – 몇 가지 사실 오류가 있으나 전반적 맥락에는 큰 영향 없음.
- **2점: 부정확함** – 중요한 사실 오류나 근거 없는 내용이 몇 개 있어 신뢰 어렵게 함.
- **1점: 사실과 거리가 멀** – 대부분 내용이 잘못되었거나 확인되지 않은 주장으로 구성됨.

위와 같은 루브릭을 미리 정해두면 여러 평가자가 평가하더라도 **객관성을 최대한 확보**할 수 있습니다. 또한 하나의 응답에 대해 여러 지표를 각각 점수화하면, 이후 평균이나 분포를 통해 **정량적인 품질 비교**가 가능합니다. 예를 들어 **동일 질문에 대한 두 모델의 응답을 평가했다면, 정확성 5 vs 3, 일관성 5 vs 5 등의 점수로 어느 쪽이 우수한지 한눈에 비교**할 수 있습니다.

또한 객관적 평가는 자동화에도 유리합니다. 명확한 기준이 있다면, 일부 지표는 코드나 **다른 LLM을 활용**해 평가할 수도 있습니다. (예: “응답이 JSON 포맷인가?”는 프로그램으로 True/False 판단 가능). 실제로 Google 등의 평가 프레임워크에서는 모델이 **기준에 부합하는 정도**를 0~5 점수로 매기는 방식을 사용합니다. 0점은 완전히 미달, 5점은 매우 우수함을 의미하도록 정의하는 식입니다.

장점: 객관적 평가는 **명확한 기준**에 따라 이루어지므로 평가자마다 결과 차이가 적고, 결과를 **수치로 취합**하여 추세를 보거나 비교 분석하기 좋습니다. 또한 평가 기준이 구체적일수록 모델 개선 시 **어느 부분을 고쳐야 하는지** 정확히 알 수 있는 피드백이 됩니다.

단점: 모든 품질 요소를 다 수치화할 수 있는 것은 아닙니다. 응답의 매력도나 미묘한 어조 등은 점수로 환산하기 어려운 측면이 있습니다. 또한 루브릭을 잘못 설계하면 오히려 중요한 부분을 놓칠 수 있습니다. 예를 들어 **“정답과의 단어 일치율”** 같은 기준은 겉보기 정확도는 측정하지만 **의미상 정답**을 놓칠 수 있습니다. 객관적 지표만 맹신하면 자칫 **숫자로 표현되지 않는 요소를 간과**할 위험도 있습니다.

3.2. 주관적 평가 – 사용자 경험 기반 판단

주관적 평가는 별도의 세세한 기준표보다는 **평가자의 전체적인 느낀 점이나 사용자 관점의 만족도**에 기대는 방식입니다. 예를 들어 응답을 읽고 종합적으로 “매우 만족”, “보통”, “실망”과 같은 **전반적 인상으로 평가**하거나, 자유 서술로 **피드백**을 남기는 것을 말합니다. 또 한 가지 형태는 여러 응답을 두고 **선호도 투표나 랭킹**을 하는 것입니다 (예: A 응답 vs B 응답 중 어느 쪽이 더 나은가?).

주관적 평가는 사람이 지닌 **섬세한 판단 능력과 맥락 이해**를 활용한다는 장점이 있습니다. 앞서 언급한 것처럼 스타일이나 톤 등은 매우 **주관적인 요소**라 숫자로 딱 잘라 판단하기 어렵습니다. 이러한 부분은 **사람이 직접 읽었을 때의 느낌**이 중요합니다. 예를 들어 응답이 아무리 사실과 맞아도 어딘지 모르게 불친절하게 느껴진다면, 이는 **정량 지표로 잡히지 않는 문제**지만 사용자는 민감하게 반응할 수 있습니다. 사람이 직접 평가하면 이러한 **미묘한 요소**까지 고려할 수 있습니다.

방법: 주관적 평가를 진행할 때는 우선 평가자(혹은 사용자)에게 **명확한 질문**을 던지는 것이 좋습니다. 예를 들어 **“이 응답은 질문에 잘 부합한다고 느껴지나요? (예/아니오)”**, **“이 답변의 어조는 적절했나요? (서술형 의견)”** 등의 형태로 피드백을 수집할 수 있습니다. 또는 단순히 **전반적인 점수** (예: 10점 만점에 몇 점?)를 매기게 하여 종합 평가 점수를 얻을 수도 있습니다.

주의점: 주관적 평가는 **평가자마다 기준이 다를 수 있고, 일관성이 떨어질 수 있다는 점**을 유의해야 합니다. 어떤 사람은 아주 간간하게 점수를 주고, 어떤 사람은 관대하게 주는 경향이 있을 수 있습니다. 또한 평가자의 개인적 취향이나 편견이 개입되어 **편향(bias)**이 생길 우려도 있습니다. 예를 들어 특정 문체를 선호하는 평가자는 그쪽에 점수를 더 줄 수 있고, 어떤 평가는 최근 본 정보에 영향을 받을 수도 있습니다. 이러한 이유로 **동일한 응답도 누가 평가**

하느냐에 따라 결과가 달라질 수 있음을 염두에 두어야 합니다. 가능하다면 여러 명이 평가하여 평균을 내거나, 평가자 간 합의를 통해 편향을 줄이는 방법을 사용합니다.

장점: 전체적인 사용자 관점에서의 체감 품질을 알 수 있습니다. 결국 최종 사용자의 만족도가 중요하므로, 주관적 평가는 현실적인 품질 감각을 제공해줍니다. 또한 숫자로 명시되지 않은 창의성, 설득력, 친절함 등의 요소도 포착하기 좋습니다.

단점: 체계적 비교나 추적이 어렵다는 점입니다. 점수를 매겨도 평가자마다 다르고 모호성이 크므로 신뢰도 확보가 어려울 수 있습니다. 또 대량의 응답을 모두 사람이 평가하기에는 비용과 시간이 많이 소요됩니다. 이러한 단점을 보완하기 위해 최근에는 사람 대신 LLM을 활용한 평가(예: GPT-4를 심판으로 활용) 등도 연구되고 있습니다만, 이 방법 역시 완전히 객관적이지는 않습니다.

3.3. 객관적 vs 주관적 평가의 활용 및 예시

두 평가 접근법은 상호 배타적이지 않으며, 오히려 함께 활용하여 보완적인 인사이트를 얻는 것이 좋습니다. 실무에서는 대개 기준표에 따른 점수화(정량)와 자유 의견 제공(정성)을 동시에 받습니다. 예를 들어 평가자가 각 항목별로 1~5점을 매긴 후, 추가로 “개선사항이나 인상 깊은 점”을 서술하게 할 수 있습니다. 이렇게 하면 숫자 데이터도 얻고 맥락 있는 피드백도 얻을 수 있어, 응답 개선에 더욱 도움이 됩니다.

활용 예시 1: 고객 지원 챗봇의 답변 평가

- 객관적 지표: 정확성, 정책 위반 여부, 형식(템플릿) 준수 여부 등을 체크리스트로 확인 (예/아니오 또는 점수)
- 주관적 지표: 실제 고객이 이 답변을 받고 만족했을지에 대한 평가자의 느낌 (만족도 1~5 또는 서술)

활용 예시 2: 튜터봇의 학습 콘텐츠 설명 평가

- 객관적 지표: 학생 수준에 맞는 난이도인지(예/아니오), 학습 목표 달성 여부(점수)
- 주관적 지표: 설명이 이해하기 쉬웠는지, 동기 부여가 되는지 등에 대한 평가자의 주관 평가

이렇듯 상황에 따라 평가 프레임워크를 설계할 때, 루브릭처럼 명확한 기준을 세우고 점수화할 부분과, 열린 의견으로 수렴할 부분을 적절히 나누는 것이 중요합니다. 정량 평가는 변화 추적과 비교 분석에 이점을 주고, 정성 평가는 구체적 개선 아이디어를 제시해주는 경향이 있습니다. 둘을 종합하면 보다 균형 잡힌 평가가 가능합니다.

마지막으로, 어떤 지표들은 명확한 객관 평가가 가능하고 (예: “금지어 포함 여부”는 객관식 판단), 어떤 것은 평가자 주관이 불가피합니다 (예: “문장이 매끄러운가”). 이런 경우 각 지표의 성격에 따라 평가 방식을 혼용하는 유연성이 필요합니다. 예를 들어 **JSON 포맷 준수**는 자동 스크립트 채점하고, **친절한 어조**는 사람이나 LLM이 판단하도록 하는 식입니다.

4. 프롬프트 평가표 설계 실습

이제 실제로 평가를 수행하기 위한 평가표(Score Sheet)를 설계하는 방법을 알아보겠습니다.

4.1. 평가표 기본 구성

평가표는 보통 행(row) 하나가 하나의 프롬프트 응답 사례에 대응되도록 설계합니다. 그리고 열(column)들은 평가 항목이나 메타정보를 나타냅니다. 예시로 다음과 같은 열 구성을 생각할 수 있습니다:

- Test ID: 평가 사례 식별자 (번호나 응답 내용 요약)
- 프롬프트 내용: 어떤 요청인지 간략히 기록 (필요하다면)
- 모델/버전: 어떤 모델이나 프롬프트 버전으로 생성된 응답인지 명시 (예: GPT-4o-mini vs GPT-4o)
- 응답 텍스트: 평가 대상이 된 실제 모델 응답 내용 (요약하거나 링크로 참조 가능)
- 정확성 점수 (1~5)
- 일관성 점수 (1~5)
- 톤 적절성 점수 (1~5)
- 포맷 적합성 점수 (1~5) – 구조/형식 평가
- 종합 평가 또는 코멘트: 전반적인 소감이나 개선 필요한 부분 메모

이러한 표를 만들면 한 눈에 여러 응답의 평가 결과를 비교할 수 있습니다. 점수 열들은 조건부 서식 등을 적용해 색상으로 높은 점수/낮은 점수를 구분하면 시각적으로 보기 편합니다. 또한 평균, 표준편차를 구해 특정 지표에서 전반적인 성과나 편차를 볼 수도 있습니다.

4.2. 템플릿 활용

1. 평가표 파일 준비: 공유된 Google Sheet 링크를 열거나, Excel 파일을 다운로드하여 열어둡니다. 열 이름과 루브릭 설명 탭 등을 확인합니다.
2. 응답 사례 확인: 사전에 준비된 프롬프트와 그에 대한 모델 응답 사례들을 살펴봅니다. (예: 튜터봇 시나리오에서 GPT-4o-mini의 응답, GPT-4o의 응답 등)
3. 평가지표별 점수 부여: 각 응답 사례에 대해 앞서 논의한 기준에 따라 정확성, 일관성, 톤 적절성, 포맷 등 항목별로 1~5점을 매겨봅니다. 점수를 매길 때는 템플릿에 제공된 루브릭 가이드를 참고하여, 해당 점수에 부합하는지 판단합니다.
4. 코멘트 작성: 필요시 종합 코멘트 열에 점수를 준 이유나 인상적인 점, 개선 아이디어를 간략히 적습니다. (예: “사실관계는 맞지만 말투가 학생에겐 어려움” 등의 메모)

5. **반복**: 준비된 여러 응답 사례에 대해 위 작업을 반복합니다. 한 가지 프롬프트에 대한 여러 모델 응답을 평가하거나, 하나의 모델이 생성한 다양한 케이스를 평가할 수 있습니다.

4.3. 평가 결과 활용

평가표는 이후 토론과 분석에 활용됩니다. 예를 들어, 동일한 프롬프트에 대한 GPT-4o-mini와 GPT-4o 응답의 점수를 나란히 비교하면 어떤 모델이 어떤 측면에서 뛰어난지 볼 수 있습니다. 혹은 한 모델의 응답들 중 어느 지표 점수가 일관되게 낮다면, 그 부분이 개선 우선 순위임을 알 수 있습니다.

또한 Sheet의 기능을 이용해 평균 점수를 계산하거나, 필터를 걸어 특정 조건의 사례만 모아 볼 수도 있습니다. 예를 들어 “정확성 3점 이하인 응답만 필터링”하여 어떤 오류가 있었는지 모아보는 등 데이터 분석이 가능합니다. 이러한 도구 활용법도 함께 익히게 될 것입니다.

5. Colab 기반 평가 자동화 실습

대규모 응답을 효율적으로 평가하기 위해, Colab에서 파이썬을 활용한 자동화 실습도 진행합니다 (시간이 충분할 경우 선택적으로). 여기서는 OpenAI API를 통해 여러 응답을 **한꺼번에 생성 및 수집**하고, Pandas 데이터프레임으로 관리하면서 **평가 작업을 보조**하는 방법을 실습합니다.

5.1. Colab 환경 설정 및 OpenAI API 사용

Colab 노트북을 열고, 우선 OpenAI API를 사용하기 위한 환경을 설정합니다. OpenAI 라이브러리 버전 1.6 이상이 설치되어 있어야 하며, API 키를 Colab 노트북에 불러오겠습니다. 다음 코드를 사용하여 Colab에 저장된 사용자 API 키를 불러올 수 있습니다:

##실습 4.1. Google Colab 실습

```
!pip install -U openai

from google.colab import userdata
import os
import openai

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

client = openai.OpenAI()
```

위 코드로 API 키가 제대로 로드되었다면, OpenAI 클라이언트를 초기화합니다. (OpenAI 패키지를 미리 `!pip install openai` 로 설치했어야 합니다.) 클라이언트 생성과 응답 호출은 다음

과 같이 수행할 수 있습니다:

```
# 예시: GPT-4o-mini 모델로 채팅 완료 생성
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "여기에 사용자 프롬프트를 입력합니다."}]
)

# 응답 내용 추출
print(response.choices[0].message.content)
```

이렇게 Colab에서 여러 프롬프트에 대한 응답을 자동 생성하면, 평가 대상 데이터를 손쉽게 얻을 수 있습니다. 실습에서는 미리 준비된 프롬프트들과 GPT-4o-mini를 활용하여 **다수의 응답**을 생성해 둘 것입니다. 예컨대 동일한 질문에 대해 다양한 톤 지시를 주거나, 또는 GPT-4o와 GPT-4o-mini 두 모델을 각각 호출하여 응답을 얻어놓을 수 있습니다.

5.2. Pandas DataFrame을 통한 평가 관리

여러 응답 데이터를 획득했다면, 이를 Pandas **DataFrame**으로 정리하여 관리합니다. 데이터프레임은 스프레드시트와 유사하게 생각할 수 있는데, 아까 4단계에서 사용한 평가표를 **Colab 내에서 다룬다**고 보면 됩니다. 예를 들어:

##실습 4.2. Google Colab 실습

```
import pandas as pd

# 예시 응답 리스트 (실제로는 API로 얻은 결과를 사용)
prompts = ["질문1", "질문1", "질문2"]
models = ["GPT-4o-mini", "GPT-4o-mini", "GPT-3.5-Turbo"]
responses = ["응답 내용 A...", "응답 내용 B...", "응답 내용 C..."]

# 데이터프레임 구성
df = pd.DataFrame({
    "prompt": prompts,
    "model": models,
    "response": responses,
    "accuracy": [None, None, None],
    "consistency": [None, None, None],
    "tone": [None, None, None],
```

```
"format": [None, None, None],
"comments": ["", "", ""]
})
```

위와 같이 DataFrame을 만들면, 각 열이 평가 항목에 대응됩니다. 초기에는 점수 칸을 `None` 이나 빈칸으로 두고, **평가자가 수기로 점수를 입력**할 수 있게 합니다. Colab에서는 DataFrame을 출력하면 표 형태로 보이므로, 수강생들은 해당 셀을 복사하여 직접 점수를 채워넣거나, 편의를 위해 CSV로 내려받아 Excel에서 채운 후 다시 불러올 수도 있습니다. (시간 관계상 Colab 상에서 직접 편집하지 않고, Google Sheet에서 평가한 결과를 CSV로 저장해 읽어오는 방법을 더 권장할 수 있습니다.)

점수가 채워진 DataFrame을 얻으면, 이후 파이썬으로 다양한 분석을 해볼 수 있습니다. 예를 들어:

```
# 각 지표별 평균 점수
print(df[["accuracy","consistency","tone","format"]].mean())

# 특정 조건 필터링: 정확성 3 이하인 사례 출력
print(df[df["accuracy"] <= 3][["model","prompt","accuracy","response"]])
```

이런 식으로 코드 몇 줄로 **평가 통계치**를 구하거나 **특정 패턴**을 찾아볼 수 있습니다. 또한 `matplotlib` 등을 사용하면 막대그래프 등으로 모델별 평균 점수를 시각화하여, 어느 모델이 전반적으로 더 나은지 한눈에 보여줄 수도 있습니다. 예를 들어 GPT-4o-mini vs GPT-4o의 지표별 점수를 그래프로 비교하면 차이를 쉽게 인식할 수 있습니다.

이번 실습의 목적은 모든 평가를 완전 자동화하는 것이 아니라, **사람의 평가 결과를 체계적으로 수집하고 분석하는 흐름**을 체험하는 것입니다. 따라서 Colab에서는 **데이터 수집 및 처리** 부분을 담당하고, **실제 평가는 수강생들이 수기로** 하지만 그 결과를 빠르게 집계/분석하는 과정을 밟게 됩니다. 향후에는 고급 주제로서 LLM을 활용한 자동 평가(예: GPT를 평정자로 활용)까지도 다룰 수 있지만, 이번에는 기본 틀에 집중합니다.

6. 실습 시나리오 예시

평가 연습을 위해 **여러 시나리오의 프롬프트와 응답 사례**를 준비했습니다. 동일한 프롬프트에 대해 **역할, 톤, 출력 포맷**을 다양하게 달리하여 응답을 생성했고, 수강생들은 이를 평가하게 됩니다. 주요 시나리오는 다음과 같습니다:

- **시나리오 1: 튜터봇** – 예를 들어 초등학생을 위한 과학 개념 설명 요청 프롬프트에 대해,
 - 답변 A: GPT-4o-mini가 **친절한 선생님** 톤으로 설명한 응답
 - 답변 B: GPT-4o-mini가 **대학 교수**처럼 다소 어려운 톤으로 설명한 응답

- 답변 C: GPT-3.5-turbo가 **친절한 선생님** 톤으로 설명한 응답

여기서 학생들은 각 응답을 정확성, 일관성, 톤 적절성, 형식 측면에서 평가하고 비교합니다. 예컨대 답변 B는 톤 적절성 점수가 낮을 수 있고, 답변 A와 C는 모델 성능 차이에 따른 정확성 또는 상세도 차이가 있을 수 있습니다.

- **시나리오 2: 고객 상담 챗봇 – “환불 정책 문의”**와 같은 프롬프트에 대해,

- 답변 A: 회사 **공식 톤(격식체)**으로 작성된 응답
- 답변 B: **지나치게 캐주얼한 톤**으로 작성된 응답
- 답변 C: 공식 톤이지만 **중요 세부 정보가 누락된** 응답 (사실성 부족)

이런 사례를 평가함으로써 어떤 답변이 고객에게 가장 적합한지, 각 답변의 부족한 점이 무엇인지 토론합니다. 예를 들어 B는 톤 부적절성으로 감점을, C는 정확성/완전성 부족으로 감점을 받을 것입니다.

- **시나리오 3: 일정 관리 비서 – “내일 일정 알려줘”**라는 동일 프롬프트에 대해,

- 답변 A: **간결한 bullet 포인트** 형식으로 일정 요약 (포맷 준수 우수)
- 답변 B: **장문의 문단** 형식으로 일정 설명 (포맷 지시에 어긋남)
- 답변 C: bullet 포맷이지만 **오류**가 섞인 일정 안내 (정확성 문제)

이 사례에서는 포맷/구조의 중요성을 체감할 수 있습니다. 학생들은 A와 B를 비교하여 형식 일관성이 사용자 경험에 미치는 영향, B와 C를 비교하여 정확성 vs 형식의 트레이드오프 등을 고민해볼 수 있습니다.

이 외에도 강의자는 추가로 **다양한 도메인**의 프롬프트 결과물을 준비해 둘 것입니다. 예를 들어, **창작 스토리 생성** 프롬프트에 대한 응답들을 평가하면서 사실성보다는 창의성이나 스토리 일관성 같은 다른 지표를 생각해볼게 할 수도 있습니다. 또는 **코딩 어시스턴트** 시나리오에서, 코드 포맷의 정확성, 오류 여부 등을 평가할 수도 있습니다.

중요한 것은, 수강생들이 **동일하거나 유사한 프롬프트의 여러 응답**을 비교 평가함으로써 어떤 차이가 발생하는지 직접 분석해보는 경험을 쌓는 것입니다. 이를 통해 모델의 동작 특성과 프롬프트 설계의 효과를 자연스럽게 익히게 됩니다.

7. 실습 결과 정리 및 응답 개선 방향

모든 실습을 마친 후에는, **평가 결과를 함께 검토**하고 **개선 방향을 논의**하는 시간을 갖습니다. 이 부분에서는 다음과 같은 활동이 이루어집니다:

- **평가 결과 공유:** 수강생들이 작성한 평가표를 바탕으로, 흥미로운 사례나 특이점에 대해 발표하거나 의견을 나눕니다. 예를 들어 “튜터봇 시나리오에서 GPT-4o-mini의 응답은 정확성은 높았지만 톤이 학생에게 다소 어려웠다”라거나 “고객 상담 시나리오에서 격식

체가 확실히 점수를 높게 받았다"와 같은 발견을 공유합니다. 다양한 관점의 평가를 듣고 **공통된 평가 패턴**을 찾아봅니다.

- **모델/프롬프트 비교 분석:** 만약 두 개 이상의 모델로 응답을 생성해봤다면, **모델 간 특성 차이**를 분석합니다. 예를 들어 GPT-4o-mini vs GPT-4o의 경우 사실 정확도에서 차이가 있었다거나, 톤 일관성은 비슷했는지 등을 토론했습니다. 또는 프롬프트에 제공한 지시 (예: "표로 출력")를 지켰는지 비율을 비교하여, 어느 설정이 **지시 준수**를 잘 하는지도 살펴봅니다.
- **낮은 점수 항목에 대한 원인 파악:** 평가 결과 특정 지표의 점수가 낮았던 응답에 대해 **왜 그런 결과가 되었는지** 원인을 분석합니다. 예를 들어 정확성 점수가 낮았던 응답은 어떤 잘못된 정보를 언급했는지, 톤 적절성 점수가 낮았다면 어떤 표현이 문제였는지 등 실제 사례를 화면에 띄워놓고 짚어봅니다. 이 과정에서 수강생들은 **구체적인 문제 사례**를 다시 한 번 확인하고, 잘 된 응답과 비교하면서 배울 점을 도출합니다.
- **응답 개선 방향 논의:** 마지막으로, 발견된 문제점을 바탕으로 **어떻게 하면 응답 품질을 개선할지 brainstorming**을 합니다. 이는 곧 **프롬프트 엔지니어링 개선 전략**과 연결됩니다. 예를 들어:
 - 사실 오류가 잦은 경우: "프롬프트에 추가로 근거를 제시하도록 요구하거나, 최신 정보를 넣어주면 좋겠다" 혹은 "GPT-4o-mini 대신 GPT-4o를 쓰니 정확성이 나아졌다" 등의 논의.
 - 톤 문제가 있는 경우: "사용자 프로필(연령 등)을 프롬프트에 명시적으로 주면 톤이 개선될 것 같다" 또는 "일관된 말투 유지를 위해 프롬프트 시작에 역할을 강조해야겠다" 등의 아이디어.
 - 형식 일관성 문제가 있는 경우: "답변 예시를 프롬프트에 제시하여 그 형식을 따라하게 하면 어떨까" 또는 "채점 스크립트로 포맷 오류를 잡아내 피드백을 주자" 등.

마지막으로, 이번 세션을 통해 배운 것을 요약하면 다음과 같습니다:

- **평가의 중요성:** LLM 응답 품질 향상을 위해 체계적 평가가 필수이며, 이를 통해 문제 원인을 파악하고 지속 개선이 가능함.
- **평가지표의 활용:** 정확성, 일관성, 톤, 형식 등 다양한 지표를 고루 살펴야 하며, 과제에 맞게 지표를 추가/조정할 수 있음.
- **평가 방법론:** 객관적 루브릭과 주관적 평가를 적절히 병행하고, 필요하면 LLM 도움을 받아 평가를 확장하는 등 현실적인 접근을 취해야 함.
- **실습 통찰:** 다양한 사례 평가를 통해 모델의 한계와 강점을 파악했고, 이를 토대로 더 나은 프롬프트 작성 전략을 생각해봄.